

```

1  """
2  Smart Patio Shield: Inference Service
3  =====
4  =====
5  A minimal FastAPI service that loads the trained
   XGBoost model (Model 1) and
6  returns a wind-driven-rain probability plus a deploy-
   or-stow decision.
7
8  Why Model 1 alone:
9      The project's central finding was that satellite
   imagery adds no discriminative
10     value for this task beyond the free reanalysis
   weather features (Section 8-9 of
11     the technical documentation). The deployed
   service therefore runs the tabular
12     model only - a deliberate engineering decision
   that follows directly from the
13     research result, and which keeps the live system
   lightweight (no satellite
14     pipeline, no GPU).
15
16  How to run:
17     pip install fastapi uvicorn xgboost pandas numpy
18     python smart-patio-shield app.py
19     # then open dashboard.html in a browser (it calls
   http://127.0.0.1:8000)
20
21  Endpoints:
22     GET /health          -> {"status": "ok", "
   model_loaded": true}
23     POST /predict        -> takes intuitive weather
   inputs, returns decision
24     GET /sample_hours    -> a handful of real test-
   set hours for the demo picker
25  """
26
27  import json
28  import math
29  from pathlib import Path

```

```

30
31 import numpy as np
32 import pandas as pd
33 import xgboost as xgb
34 from fastapi import FastAPI
35 from fastapi.middleware.cors import CORSMiddleware
36 from pydantic import BaseModel
37
38 # -----
39 # Paths
40 # -----
41 HERE = Path(__file__).resolve().parent
42 MODEL_PATH = HERE / "xgboost_baseline.json"
43 MANIFEST_PATH = HERE / "baseline_training.manifest.
  json"
44 SAMPLE_PATH = HERE / "sample_hours.json"
  # generated demo hours from real test set
45
46 # The exact feature schema the model was trained on (
  from baseline.py)
47 NUMERIC_FEATURES = [
48     "temperature_2m", "relative_humidity_2m", "
  dew_point_2m", "apparent_temperature",
49     "pressure_msl", "surface_pressure",
50     "cloud_cover", "cloud_cover_low", "
  cloud_cover_mid", "cloud_cover_high",
51     "wind_speed_10m",
52     "et0_fao_evapotranspiration", "
  vapour_pressure_deficit", "sunshine_duration",
53     "oni",
54     "hour", "month", "hour_sin", "hour_cos", "
  month_sin", "month_cos",
55     "precip_lag1", "precip_lag3", "precip_sum_3h",
56     "gust_lag1", "gust_max_3h",
57     "pressure_msl_trend_3h", "pressure_msl_trend_6h",
58     "cloud_cover_low_lag1",
59     "humidity_lag1", "humidity_lag3",
60 ]
61 CATEGORICAL_FEATURES = ["location", "enso_phase"]

```

```

62 LOCATIONS = ["kingston", "montego_bay"]
63 ENSO_PHASES = ["el_nino", "la_nina", "neutral"]
64
65 # Decision threshold - the operating point for
    deploy/stow.
66 # 0.5 is the default; a real deployment would tune
    this to the cost of a
67 # missed event vs a false alarm (see technical doc
    Section 7.4).
68 DECISION_THRESHOLD = 0.5
69
70 app = FastAPI(title="Smart Patio Shield Inference
    Service")
71 app.add_middleware(
72     CORSMiddleware, allow_origins=["*"],
    allow_methods=["*"], allow_headers=["*"],
73 )
74
75 # -----
    -----
76 # Load model + the exact post-dummy column order the
    booster expects
77 # -----
    -----
78 _model = None
79 _fit_columns = None
80
81
82 def _load():
83     global _model, _fit_columns
84     _model = xgb.XGBClassifier()
85     _model.load_model(str(MODEL_PATH))
86     if MANIFEST_PATH.exists():
87         _fit_columns = json.load(open(MANIFEST_PATH
    ))["feature_columns"]
88     else:
89         # Reconstruct the dummy-expanded column
    order deterministically
90         cols = list(NUMERIC_FEATURES)
91         for loc in LOCATIONS:
92             cols.append(f"location_{loc}")

```

```

93         for ph in ENSO_PHASES:
94             cols.append(f"enso_phase_{ph}")
95             _fit_columns = cols
96             print(f"Model loaded. Expecting {len(
_fit_columns)} columns.")
97
98
99 @app.on_event("startup")
100 def startup():
101     _load()
102
103
104 # -----
-----
105 # Input model - Intuitive controls.
106 # The service derives the full 30+ feature vector
from these, so the user manipulates human-
understandable quantities.
107 # -----
-----
108 class WeatherInput(BaseModel):
109     location: str = "montego_bay"          #
kingston | montego_bay
110     hour: int = 14                        # 0-23
local
111     month: int = 10                       # 1-12
112     relative_humidity: float = 85.0       # %
113     cloud_cover_low: float = 70.0         # %
114     cloud_cover_mid: float = 50.0         # %
115     cloud_cover_high: float = 30.0        # %
116     recent_rain_mm: float = 0.5           # mm in
the previous hour (precip_lag1)
117     recent_gust_kmh: float = 25.0         # max
gust previous hour (gust_lag1)
118     pressure_falling: bool = True         # is
pressure trending down (storm sign)?
119     enso_phase: str = "neutral"          #
el_nino | la_nina | neutral
120     threshold: float = DECISION_THRESHOLD # allow
the demo to move the operating point
121

```

```

122
123 def _derive_features(w: WeatherInput) -> pd.
    DataFrame:
124     """Turn the ~11 intuitive inputs into the full
        model feature row.
125
126     Non-exposed features are filled with physically
        reasonable values derived
127     from the exposed ones (e.g. dew point from
        humidity and temperature).
128     """
129     # crude but reasonable temperature by hour (
        tropical coastal diurnal range)
130     temp = 26.0 + 4.0 * math.sin((w.hour - 9) / 24
        * 2 * math.pi)
131     rh = float(np.clip(w.relative_humidity, 1, 100))
132     # dew point via Magnus approximation
133     a, b = 17.27, 237.7
134     gamma = (a * temp) / (b + temp) + math.log(rh /
        100.0)
135     dew = (b * gamma) / (a - gamma)
136     cloud_all = np.mean([w.cloud_cover_low, w.
        cloud_cover_mid, w.cloud_cover_high])
137     # vapour pressure deficit (kPa), rough
138     es = 0.6108 * math.exp((17.27 * temp) / (temp +
        237.3))
139     vpd = es * (1 - rh / 100.0)
140     press_trend = -0.9 if w.pressure_falling else 0.
        4 # hPa over 3h, sign is what matters
141
142     row = {
143         "temperature_2m": temp,
144         "relative_humidity_2m": rh,
145         "dew_point_2m": dew,
146         "apparent_temperature": temp - 1.0,
147         "pressure_msl": 1013.0 + press_trend,
148         "surface_pressure": 1011.0 + press_trend,
149         "cloud_cover": cloud_all,
150         "cloud_cover_low": w.cloud_cover_low,
151         "cloud_cover_mid": w.cloud_cover_mid,
152         "cloud_cover_high": w.cloud_cover_high,

```

```

153         "wind_speed_10m": max(6.0, w.recent_gust_kmh
    * 0.6),
154         "et0_fao_evapotranspiration": max(0.0, 0.15
    * (1 - cloud_all / 100)),
155         "vapour_pressure_deficit": max(0.0, vpd),
156         "sunshine_duration": max(0.0, 3600 * (1 -
cloud_all / 100) * (1 if 6 <= w.hour <= 18 else 0)),
157         "oni": {"la_nina": -1.0, "neutral": 0.0, "
el_nino": 1.0}[w.enso_phase],
158         "hour": w.hour,
159         "month": w.month,
160         "hour_sin": math.sin(2 * math.pi * w.hour /
    24),
161         "hour_cos": math.cos(2 * math.pi * w.hour /
    24),
162         "month_sin": math.sin(2 * math.pi * w.month
    / 12),
163         "month_cos": math.cos(2 * math.pi * w.month
    / 12),
164         "precip_lag1": w.recent_rain_mm,
165         "precip_lag3": w.recent_rain_mm * 1.5,
166         "precip_sum_3h": w.recent_rain_mm * 2.0,
167         "gust_lag1": w.recent_gust_kmh,
168         "gust_max_3h": w.recent_gust_kmh * 1.1,
169         "pressure_msl_trend_3h": press_trend,
170         "pressure_msl_trend_6h": press_trend * 1.6,
171         "cloud_cover_low_lag1": w.cloud_cover_low,
172         "humidity_lag1": rh,
173         "humidity_lag3": rh,
174     }
175     # one-hot the categoricals to match training
176     for loc in LOCATIONS:
177         row[f"location_{loc}"] = 1.0 if w.location
    == loc else 0.0
178     for ph in ENSO_PHASES:
179         row[f"enso_phase_{ph}"] = 1.0 if w.
enso_phase == ph else 0.0
180
181     X = pd.DataFrame([row])
182     for c in _fit_columns:
183         if c not in X.columns:

```

```

184         X[c] = 0.0
185     return X[_fit_columns]
186
187
188 @app.get("/health")
189 def health():
190     return {"status": "ok", "model_loaded": _model
191           is not None,
192           "n_features": len(_fit_columns) if
193           _fit_columns else 0}
194
195 @app.post("/predict")
196 def predict(w: WeatherInput):
197     X = _derive_features(w)
198     prob = float(_model.predict_proba(X)[: , 1][0])
199     thr = float(w.threshold)
200     decision = "DEPLOY" if prob >= thr else "STOW"
201     drivers = []
202     if w.recent_rain_mm > 0.5:
203         drivers.append("recent rain (strong
204         persistence signal)")
205     if w.cloud_cover_low > 60:
206         drivers.append("heavy low cloud")
207     if w.recent_gust_kmh > 25:
208         drivers.append("gusty wind")
209     if w.pressure_falling:
210         drivers.append("falling pressure")
211     if 12 <= w.hour <= 18:
212         drivers.append("afternoon convection window")
213
214     return {
215         "probability": round(prob, 4),
216         "threshold": thr,
217         "decision": decision,
218         "confidence_band": ("high" if abs(prob - thr
219         ) > 0.25 else "borderline"),
220         "key_drivers": drivers or ["conditions
221         generally dry"],
222     }

```

```
219
220 @app.get("/sample_hours")
221 def sample_hours():
222     if SAMPLE_PATH.exists():
223         return json.load(open(SAMPLE_PATH))
224     return {"hours": [], "note": "sample_hours.json
not generated"}
225
226
227 if __name__ == "__main__":
228     import uvicorn
229     uvicorn.run(app, host="127.0.0.1", port=8000)
230
```